

Pygame Zombie Game Implementation

<https://gemini.google.com/app/e2c810139d017c55>

User prompt: I have a Pygame script (game.py). Please modify it to implement both Task 1 and Task 2: Health System: Give the player 3 HP (hp = 3). Taking a hit from a zombie subtracts 1 HP and triggers 60 frames (1 second) of invincibility with a visual flicker effect to prevent instant death. Show HP: {hp} in the top HUD. Trigger Game Over only when HP reaches 0. Ammo System: Limit the player to 12 bullets per clip. Block shooting at 0 ammo or during reloading. Trigger a 2-second reload timer (120 frames) automatically when attempting to shoot at 0 ammo, or manually when pressing R. Display ammo state (Ammo: 12/12 or RELOADING...) in the top HUD. First give task 1 implemented code then task 2 code implements along with Task 1

Response: Here is the code with **Task 1** implemented first, followed by the combined code implementing both **Task 1** and **Task 2**.

Implementation 1: Task 1 Only (Health System)

```
import pygame
import random
import math
import time

WIDTH, HEIGHT = 800, 560
FPS = 60
BG = (30, 35, 25)

class Zombie:
    def __init__(self, x, y, z_type="normal"):
        self.type = z_type
        if z_type == "fast":
            self.rect = pygame.Rect(x, y, 22, 22)
            self.color = (200, 180, 50) # Yellow
            self.hp = 1
            self.speed = 2.8
        elif z_type == "tank":
            self.rect = pygame.Rect(x, y, 42, 42)
            self.color = (140, 50, 50) # Dark Red
            self.hp = 6
            self.speed = 0.8
        else: # normal
            self.rect = pygame.Rect(x, y, 30, 30)
            self.color = (60, 140, 60) # Green
            self.hp = 3
            self.speed = 1.5

        self.wobble = random.uniform(0, 6.28)
        self.frame = 0

    def update(self, player_pos):
        px, py = player_pos
        cx, cy = self.rect.center
        dx, dy = px - cx, py - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist:
            self.rect.x += int(dx / dist * self.speed)
            self.rect.y += int(dy / dist * self.speed)
        self.frame += 1

    def hit(self):
        self.hp -= 1
        return self.hp <= 0

    def draw(self, screen):
        wobble_y = int(math.sin(self.frame * 0.2) * 3)
        draw_rect = self.rect.move(0, wobble_y)
        pygame.draw.rect(screen, self.color, draw_rect, border_radius=5)

        eye_offset = 4 if self.type == "fast" else (8 if self.type == "tank" else 6)
        for ex in [draw_rect.x + eye_offset, draw_rect.x + draw_rect.width - eye_offset - 4]:
            pygame.draw.circle(screen, (200, 40, 40), (ex, draw_rect.y + 8), 3 if self.type == "fast" else 4)

class Barrel:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.color = (180, 90, 30)
        self.radius = 120

    def draw(self, screen):
        pygame.draw.rect(screen, self.color, self.rect, border_radius=4)
        pygame.draw.rect(screen, (220, 140, 40), self.rect, width=2, border_radius=4)
```

```

def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width - 30)
        y = random.randint(0, height - 30)
        rect = pygame.Rect(x, y, 30, 30)
        if not rect.colliderect(player_rect.inflate(margin, margin)):
            z_type = random.choices(["normal", "fast", "tank"], weights=[0.5, 0.3, 0.2])[0]
            return Zombie(x, y, z_type)

SPEED = 4

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60, 160, 220)
        self.bullets = []
        self.shoot_cooldown = 0
        self.hp = 3
        self.invincible_timer = 0

    def move(self, keys, width, height):
        dx = dy = 0
        if keys[pygame.K_w] or keys[pygame.K_UP]: dy = -SPEED
        if keys[pygame.K_s] or keys[pygame.K_DOWN]: dy = SPEED
        if keys[pygame.K_a] or keys[pygame.K_LEFT]: dx = -SPEED
        if keys[pygame.K_d] or keys[pygame.K_RIGHT]: dx = SPEED
        self.rect.x = max(0, min(width - self.rect.width, self.rect.x + dx))
        self.rect.y = max(0, min(height - self.rect.height, self.rect.y + dy))

        if self.shoot_cooldown > 0:
            self.shoot_cooldown -= 1
        if self.invincible_timer > 0:
            self.invincible_timer -= 1

    def take_damage(self):
        if self.invincible_timer == 0:
            self.hp -= 1
            self.invincible_timer = 60
            return True
        return False

    def shoot(self, target_pos):
        if self.shoot_cooldown > 0: return

        cx, cy = self.rect.center
        tx, ty = target_pos
        dx, dy = tx - cx, ty - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist == 0: return
        vx, vy = dx / dist * 10, dy / dist * 10
        self.bullets.append([cx - 4, cy - 4, vx, vy])
        self.shoot_cooldown = 15

    def update_bullets(self, width, height):
        live = []
        for b in self.bullets:
            b[0] += b[2]; b[1] += b[3]
            if 0 <= b[0] <= width and 0 <= b[1] <= height:
                live.append(b)
        self.bullets = live

    def draw(self, screen):
        if self.invincible_timer % 10 < 5:
            pygame.draw.rect(screen, self.color, self.rect, border_radius=6)

        for b in self.bullets:
            pygame.draw.circle(screen, (255, 220, 60), (int(b[0]), int(b[1])), 5)

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Zombie Escape")
        self.clock = pygame.time.Clock()

```

```

self.font = pygame.font.SysFont("monospace", 18)
self.big_font = pygame.font.SysFont("monospace", 44, bold=True)
self.explosions = []
self.reset()

def reset(self):
    self.player = Player(WIDTH // 2, HEIGHT // 2)
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
    self.barrels = [
        Barrel(150, 120),
        Barrel(650, 120),
        Barrel(150, 420),
        Barrel(650, 420)
    ]
    self.score = 0
    self.wave = 1
    self.kills = 0
    self.kills_to_next = 8
    self.game_over = False
    self.start_time = time.time()
    self.explosions = []

def trigger_explosion(self, cx, cy, radius):
    self.explosions.append({'pos': (cx, cy), 'radius': radius, 'duration': 15})
    dead_zombies = []
    for z in self.zombies:
        zx, zy = z.rect.center
        dist = math.hypot(zx - cx, zy - cy)
        if dist <= radius:
            dead_zombies.append(z)

    for z in dead_zombies:
        if z in self.zombies:
            self.zombies.remove(z)
            self.kills += 1
            self.score += 10

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r and self.game_over:
                self.reset()
        if event.type == pygame.MOUSEBUTTONDOWN and not self.game_over:
            self.player.shoot(event.pos)
    return True

def update(self):
    if self.game_over: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, WIDTH, HEIGHT)
    self.player.update_bullets(WIDTH, HEIGHT)
    self.score = int(time.time() - self.start_time)

    self.explosions = [e for e in self.explosions if e['duration'] > 0]
    for e in self.explosions:
        e['duration'] -= 1

    for z in self.zombies:
        z.update(self.player.rect.center)
        if z.rect.colliderect(self.player.rect):
            if self.player.take_damage():
                if self.player.hp <= 0:
                    self.game_over = True

    dead = []
    barrels_to_remove = []
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])

        for barrel in self.barrels:
            if barrel.rect.collidepoint(bx, by):
                if b in self.player.bullets:
                    self.player.bullets.remove(b)
                if barrel not in barrels_to_remove:
                    barrels_to_remove.append(barrel)
                self.trigger_explosion(barrel.rect.centerx, barrel.rect.centery, barrel.radius)
                break

```

```

        for z in self.zombies:
            if z.rect.collidepoint(bx, by):
                if z.hit():
                    dead.append(z)
                if b in self.player.bullets:
                    self.player.bullets.remove(b)

    for barrel in barrels_to_remove:
        if barrel in self.barrels:
            self.barrels.remove(barrel)

    for z in dead:
        if z in self.zombies:
            self.zombies.remove(z)
            self.kills += 1
            self.score += 10

    if self.kills >= self.kills_to_next:
        self.kills = 0
        self.wave += 1
        self.kills_to_next = 8 + self.wave * 2
        for _ in range(self.wave + 3):
            self.zombies.append(spawn_zombie(WIDTH, HEIGHT, self.player.rect))

def draw(self):
    self.screen.fill(BG)
    for x in range(0, WIDTH, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (x, 0), (x, HEIGHT), 1)
    for y in range(0, HEIGHT, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (0, y), (WIDTH, y), 1)

    for barrel in self.barrels:
        barrel.draw(self.screen)
    for z in self.zombies:
        z.draw(self.screen)
    self.player.draw(self.screen)

    for e in self.explosions:
        pygame.draw.circle(self.screen, (255, 120, 30), e['pos'], e['radius'], 3)

    hud_bg = pygame.Rect(0, 0, WIDTH, 40)
    pygame.draw.rect(self.screen, (15, 20, 15), hud_bg)

    hud = self.font.render(
        f"HP: {self.player.hp} Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.kills_to_next}"
        True, (160, 220, 120))
    self.screen.blit(hud, (8, 10))

    if self.game_over:
        ov = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        ov.fill((0, 0, 0, 160))
        self.screen.blit(ov, (0, 0))
        m = self.big_font.render("DEVoured!", True, (180, 40, 40))
        s = self.font.render(f"Wave {self.wave} | Score {self.score} | Press R to Restart", True, (200, 200, 200))
        self.screen.blit(m, (WIDTH // 2 - m.get_width() // 2, HEIGHT // 2 - 40))
        self.screen.blit(s, (WIDTH // 2 - s.get_width() // 2, HEIGHT // 2 + 20))
        pygame.display.flip()

def run(self):
    running = True
    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()

```

Implementation 2: Task 1 + Task 2 Combined

```

import pygame
import random
import math
import time

```

```

WIDTH, HEIGHT = 800, 560
FPS = 60
BG = (30, 35, 25)

```

```

class Zombie:
    def __init__(self, x, y, z_type="normal"):
        self.type = z_type
        if z_type == "fast":
            self.rect = pygame.Rect(x, y, 22, 22)
            self.color = (200, 180, 50) # Yellow
            self.hp = 1
            self.speed = 2.8
        elif z_type == "tank":
            self.rect = pygame.Rect(x, y, 42, 42)
            self.color = (140, 50, 50) # Dark Red
            self.hp = 6
            self.speed = 0.8
        else: # normal
            self.rect = pygame.Rect(x, y, 30, 30)
            self.color = (60, 140, 60) # Green
            self.hp = 3
            self.speed = 1.5

        self.wobble = random.uniform(0, 6.28)
        self.frame = 0

    def update(self, player_pos):
        px, py = player_pos
        cx, cy = self.rect.center
        dx, dy = px - cx, py - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist:
            self.rect.x += int(dx / dist * self.speed)
            self.rect.y += int(dy / dist * self.speed)
        self.frame += 1

    def hit(self):
        self.hp -= 1
        return self.hp <= 0

    def draw(self, screen):
        wobble_y = int(math.sin(self.frame * 0.2) * 3)
        draw_rect = self.rect.move(0, wobble_y)
        pygame.draw.rect(screen, self.color, draw_rect, border_radius=5)

        eye_offset = 4 if self.type == "fast" else (8 if self.type == "tank" else 6)
        for ex in [draw_rect.x + eye_offset, draw_rect.x + draw_rect.width - eye_offset - 4]:
            pygame.draw.circle(screen, (200, 40, 40), (ex, draw_rect.y + 8), 3 if self.type == "fast" else 4)

```

```

class Barrel:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.color = (180, 90, 30)
        self.radius = 120

    def draw(self, screen):
        pygame.draw.rect(screen, self.color, self.rect, border_radius=4)
        pygame.draw.rect(screen, (220, 140, 40), self.rect, width=2, border_radius=4)

def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width - 30)
        y = random.randint(0, height - 30)
        rect = pygame.Rect(x, y, 30, 30)
        if not rect.colliderect(player_rect.inflate(margin, margin)):
            z_type = random.choices(["normal", "fast", "tank"], weights=[0.5, 0.3, 0.2])[0]
            return Zombie(x, y, z_type)

```

```

SPEED = 4

```

```

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60, 160, 220)

```

```

        self.bullets = []
        self.shoot_cooldown = 0
        self.hp = 3
        self.invincible_timer = 0
        self.max_ammo = 12
        self.ammo = 12
        self.reload = False
        self.reload_timer = 0

    def move(self, keys, width, height):
        dx = dy = 0
        if keys[pygame.K_w] or keys[pygame.K_UP]: dy = -SPEED
        if keys[pygame.K_s] or keys[pygame.K_DOWN]: dy = SPEED
        if keys[pygame.K_a] or keys[pygame.K_LEFT]: dx = -SPEED
        if keys[pygame.K_d] or keys[pygame.K_RIGHT]: dx = SPEED
        self.rect.x = max(0, min(width - self.rect.width, self.rect.x + dx))
        self.rect.y = max(0, min(height - self.rect.height, self.rect.y + dy))

        if self.shoot_cooldown > 0:
            self.shoot_cooldown -= 1
        if self.invincible_timer > 0:
            self.invincible_timer -= 1

        if self.reload:
            self.reload_timer -= 1
            if self.reload_timer <= 0:
                self.ammo = self.max_ammo
                self.reload = False

    def reload(self):
        if not self.reload and self.ammo < self.max_ammo:
            self.reload = True
            self.reload_timer = 120

    def take_damage(self):
        if self.invincible_timer == 0:
            self.hp -= 1
            self.invincible_timer = 60
            return True
        return False

    def shoot(self, target_pos):
        if self.shoot_cooldown > 0 or self.reload: return
        if self.ammo <= 0:
            self.reload()
            return

        cx, cy = self.rect.center
        tx, ty = target_pos
        dx, dy = tx - cx, ty - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist == 0: return
        vx, vy = dx / dist * 10, dy / dist * 10
        self.bullets.append([cx - 4, cy - 4, vx, vy])
        self.shoot_cooldown = 15
        self.ammo -= 1

    def update_bullets(self, width, height):
        live = []
        for b in self.bullets:
            b[0] += b[2]; b[1] += b[3]
            if 0 <= b[0] <= width and 0 <= b[1] <= height:
                live.append(b)
        self.bullets = live

    def draw(self, screen):
        if self.invincible_timer % 10 < 5:
            pygame.draw.rect(screen, self.color, self.rect, border_radius=6)

        for b in self.bullets:
            pygame.draw.circle(screen, (255, 220, 60), (int(b[0]), int(b[1])), 5)

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Zombie Escape")
        self.clock = pygame.time.Clock()

```

```

self.font = pygame.font.SysFont("monospace", 18)
self.big_font = pygame.font.SysFont("monospace", 44, bold=True)
self.explosions = []
self.reset()

def reset(self):
    self.player = Player(WIDTH // 2, HEIGHT // 2)
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
    self.barrels = [
        Barrel(150, 120),
        Barrel(650, 120),
        Barrel(150, 420),
        Barrel(650, 420)
    ]
    self.score = 0
    self.wave = 1
    self.kills = 0
    self.kills_to_next = 8
    self.game_over = False
    self.start_time = time.time()
    self.explosions = []

def trigger_explosion(self, cx, cy, radius):
    self.explosions.append({'pos': (cx, cy), 'radius': radius, 'duration': 15})
    dead_zombies = []
    for z in self.zombies:
        zx, zy = z.rect.center
        dist = math.hypot(zx - cx, zy - cy)
        if dist <= radius:
            dead_zombies.append(z)

    for z in dead_zombies:
        if z in self.zombies:
            self.zombies.remove(z)
            self.kills += 1
            self.score += 10

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                if self.game_over:
                    self.reset()
                else:
                    self.player.reload()
            if event.type == pygame.MOUSEBUTTONDOWN and not self.game_over:
                self.player.shoot(event.pos)
    return True

def update(self):
    if self.game_over: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, WIDTH, HEIGHT)
    self.player.update_bullets(WIDTH, HEIGHT)
    self.score = int(time.time() - self.start_time)

    self.explosions = [e for e in self.explosions if e['duration'] > 0]
    for e in self.explosions:
        e['duration'] -= 1

    for z in self.zombies:
        z.update(self.player.rect.center)
        if z.rect.colliderect(self.player.rect):
            if self.player.take_damage():
                if self.player.hp <= 0:
                    self.game_over = True

    dead = []
    barrels_to_remove = []
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])

        for barrel in self.barrels:
            if barrel.rect.collidepoint(bx, by):
                if b in self.player.bullets:
                    self.player.bullets.remove(b)
                if barrel not in barrels_to_remove:
                    barrels_to_remove.append(barrel)

```

```

        self.trigger_explosion(barrel.rect.centerx, barrel.rect.centery, barrel.radius)
        break

    for z in self.zombies:
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
                self.player.bullets.remove(b)

    for barrel in barrels_to_remove:
        if barrel in self.barrels:
            self.barrels.remove(barrel)

    for z in dead:
        if z in self.zombies:
            self.zombies.remove(z)
            self.kills += 1
            self.score += 10

    if self.kills >= self.kills_to_next:
        self.kills = 0
        self.wave += 1
        self.kills_to_next = 8 + self.wave * 2
        for _ in range(self.wave + 3):
            self.zombies.append(spawn_zombie(WIDTH, HEIGHT, self.player.rect))

def draw(self):
    self.screen.fill(BG)
    for x in range(0, WIDTH, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (x, 0), (x, HEIGHT), 1)
    for y in range(0, HEIGHT, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (0, y), (WIDTH, y), 1)

    for barrel in self.barrels:
        barrel.draw(self.screen)
    for z in self.zombies:
        z.draw(self.screen)
    self.player.draw(self.screen)

    for e in self.explosions:
        pygame.draw.circle(self.screen, (255, 120, 30), e['pos'], e['radius'], 3)

    hud_bg = pygame.Rect(0, 0, WIDTH, 40)
    pygame.draw.rect(self.screen, (15, 20, 15), hud_bg)

    ammo_str = "RELOADING..." if self.player.reloadng else f"Ammo: {self.player.ammo}/{self.player.max_ammo}"
    hud = self.font.render(
        f"HP: {self.player.hp} {ammo_str} Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.kills_to_next}",
        True, (160, 220, 120))
    self.screen.blit(hud, (8, 10))

    if self.game_over:
        ov = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        ov.fill((0, 0, 0, 160))
        self.screen.blit(ov, (0, 0))
        m = self.big_font.render("DEVoured!", True, (180, 40, 40))
        s = self.font.render(f"Wave {self.wave} | Score {self.score} | Press R to Restart", True, (200, 200, 200))
        self.screen.blit(m, (WIDTH // 2 - m.get_width() // 2, HEIGHT // 2 - 40))
        self.screen.blit(s, (WIDTH // 2 - s.get_width() // 2, HEIGHT // 2 + 20))
    pygame.display.flip()

def run(self):
    running = True
    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()

```

User prompt: Now add Task 3 (Explosive Barrels) to game.py: Place 4 barrels (Barrel class) at fixed positions on the map (e.g., coordinates (150, 120), (650, 120), (150, 420), (650, 420)). When a bullet hits a barrel, trigger an explosion effect with a 120px radius that destroys all zombies within range and awards kills/score. Remove the barrel from the map after it explodes.

Response: Here is the updated code incorporating **Task 3 (Explosive Barrels)** along with the existing Health and Ammo systems:

```
import pygame
import random
import math
import time

WIDTH, HEIGHT = 800, 560
FPS = 60
BG = (30, 35, 25)

class Zombie:
    def __init__(self, x, y, z_type="normal"):
        self.type = z_type
        if z_type == "fast":
            self.rect = pygame.Rect(x, y, 22, 22)
            self.color = (200, 180, 50) # Yellow
            self.hp = 1
            self.speed = 2.8
        elif z_type == "tank":
            self.rect = pygame.Rect(x, y, 42, 42)
            self.color = (140, 50, 50) # Dark Red
            self.hp = 6
            self.speed = 0.8
        else: # normal
            self.rect = pygame.Rect(x, y, 30, 30)
            self.color = (60, 140, 60) # Green
            self.hp = 3
            self.speed = 1.5

        self.wobble = random.uniform(0, 6.28)
        self.frame = 0

    def update(self, player_pos):
        px, py = player_pos
        cx, cy = self.rect.center
        dx, dy = px - cx, py - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist:
            self.rect.x += int(dx / dist * self.speed)
            self.rect.y += int(dy / dist * self.speed)
        self.frame += 1

    def hit(self):
        self.hp -= 1
        return self.hp <= 0

    def draw(self, screen):
        wobble_y = int(math.sin(self.frame * 0.2) * 3)
        draw_rect = self.rect.move(0, wobble_y)
        pygame.draw.rect(screen, self.color, draw_rect, border_radius=5)

        eye_offset = 4 if self.type == "fast" else (8 if self.type == "tank" else 6)
        for ex in [draw_rect.x + eye_offset, draw_rect.x + draw_rect.width - eye_offset - 4]:
            pygame.draw.circle(screen, (200, 40, 40), (ex, draw_rect.y + 8), 3 if self.type == "fast" else 4)

class Barrel:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.color = (180, 90, 30)
        self.radius = 120

    def draw(self, screen):
        pygame.draw.rect(screen, self.color, self.rect, border_radius=4)
        pygame.draw.rect(screen, (220, 140, 40), self.rect, width=2, border_radius=4)

def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width - 30)
        y = random.randint(0, height - 30)
        rect = pygame.Rect(x, y, 30, 30)
        if not rect.colliderect(player_rect.inflate(margin, margin)):
            z_type = random.choices(["normal", "fast", "tank"], weights=[0.5, 0.3, 0.2])[0]
```

```
    return Zombie(x, y, z_type)
```

```
SPEED = 4
```

```
class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60, 160, 220)
        self.bullets = []
        self.shoot_cooldown = 0
        self.hp = 3
        self.invincible_timer = 0
        self.max_ammo = 12
        self.ammo = 12
        self.reload = False
        self.reload_timer = 0

    def move(self, keys, width, height):
        dx = dy = 0
        if keys[pygame.K_w] or keys[pygame.K_UP]: dy = -SPEED
        if keys[pygame.K_s] or keys[pygame.K_DOWN]: dy = SPEED
        if keys[pygame.K_a] or keys[pygame.K_LEFT]: dx = -SPEED
        if keys[pygame.K_d] or keys[pygame.K_RIGHT]: dx = SPEED
        self.rect.x = max(0, min(width - self.rect.width, self.rect.x + dx))
        self.rect.y = max(0, min(height - self.rect.height, self.rect.y + dy))

        if self.shoot_cooldown > 0:
            self.shoot_cooldown -= 1
        if self.invincible_timer > 0:
            self.invincible_timer -= 1

        if self.reload:
            self.reload_timer -= 1
            if self.reload_timer <= 0:
                self.ammo = self.max_ammo
                self.reload = False

    def reload(self):
        if not self.reload and self.ammo < self.max_ammo:
            self.reload = True
            self.reload_timer = 120

    def take_damage(self):
        if self.invincible_timer == 0:
            self.hp -= 1
            self.invincible_timer = 60
            return True
        return False

    def shoot(self, target_pos):
        if self.shoot_cooldown > 0 or self.reload: return
        if self.ammo <= 0:
            self.reload()
            return

        cx, cy = self.rect.center
        tx, ty = target_pos
        dx, dy = tx - cx, ty - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist == 0: return
        vx, vy = dx / dist * 10, dy / dist * 10
        self.bullets.append([cx - 4, cy - 4, vx, vy])
        self.shoot_cooldown = 15
        self.ammo -= 1

    def update_bullets(self, width, height):
        live = []
        for b in self.bullets:
            b[0] += b[2]; b[1] += b[3]
            if 0 <= b[0] <= width and 0 <= b[1] <= height:
                live.append(b)
        self.bullets = live

    def draw(self, screen):
        if self.invincible_timer % 10 < 5:
            pygame.draw.rect(screen, self.color, self.rect, border_radius=6)
```

```

        for b in self.bullets:
            pygame.draw.circle(screen, (255, 220, 60), (int(b[0]), int(b[1])), 5)

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Zombie Escape")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 18)
        self.big_font = pygame.font.SysFont("monospace", 44, bold=True)
        self.explosions = []
        self.reset()

    def reset(self):
        self.player = Player(WIDTH // 2, HEIGHT // 2)
        self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
        self.barrels = [
            Barrel(150, 120),
            Barrel(650, 120),
            Barrel(150, 420),
            Barrel(650, 420)
        ]
        self.score = 0
        self.wave = 1
        self.kills = 0
        self.kills_to_next = 8
        self.game_over = False
        self.start_time = time.time()
        self.explosions = []

    def trigger_explosion(self, cx, cy, radius):
        self.explosions.append({'pos': (cx, cy), 'radius': radius, 'duration': 15})
        dead_zombies = []
        for z in self.zombies:
            zx, zy = z.rect.center
            dist = math.hypot(zx - cx, zy - cy)
            if dist <= radius:
                dead_zombies.append(z)

        for z in dead_zombies:
            if z in self.zombies:
                self.zombies.remove(z)
                self.kills += 1
                self.score += 10

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    if self.game_over:
                        self.reset()
                    else:
                        self.player.reload()
            if event.type == pygame.MOUSEBUTTONDOWN and not self.game_over:
                self.player.shoot(event.pos)
        return True

    def update(self):
        if self.game_over: return
        keys = pygame.key.get_pressed()
        self.player.move(keys, WIDTH, HEIGHT)
        self.player.update_bullets(WIDTH, HEIGHT)
        self.score = int(time.time() - self.start_time)

        self.explosions = [e for e in self.explosions if e['duration'] > 0]
        for e in self.explosions:
            e['duration'] -= 1

        for z in self.zombies:
            z.update(self.player.rect.center)
            if z.rect.colliderect(self.player.rect):
                if self.player.take_damage():
                    if self.player.hp <= 0:
                        self.game_over = True

        dead = []

```

```

barrels_to_remove = []
for b in self.player.bullets[:]:
    bx, by = int(b[0]), int(b[1])

    # Check barrel collisions
    for barrel in self.barrels:
        if barrel.rect.collidepoint(bx, by):
            if b in self.player.bullets:
                self.player.bullets.remove(b)
            if barrel not in barrels_to_remove:
                barrels_to_remove.append(barrel)
            self.trigger_explosion(barrel.rect.centerx, barrel.rect.centery, barrel.radius)
            break

    # Check zombie collisions
    for z in self.zombies:
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
                self.player.bullets.remove(b)

for barrel in barrels_to_remove:
    if barrel in self.barrels:
        self.barrels.remove(barrel)

for z in dead:
    if z in self.zombies:
        self.zombies.remove(z)
        self.kills += 1
        self.score += 10

if self.kills >= self.kills_to_next:
    self.kills = 0
    self.wave += 1
    self.kills_to_next = 8 + self.wave * 2
    for _ in range(self.wave + 3):
        self.zombies.append(spawn_zombie(WIDTH, HEIGHT, self.player.rect))

def draw(self):
    self.screen.fill(BG)
    for x in range(0, WIDTH, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (x, 0), (x, HEIGHT), 1)
    for y in range(0, HEIGHT, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (0, y), (WIDTH, y), 1)

    for barrel in self.barrels:
        barrel.draw(self.screen)
    for z in self.zombies:
        z.draw(self.screen)
    self.player.draw(self.screen)

    for e in self.explosions:
        pygame.draw.circle(self.screen, (255, 120, 30), e['pos'], e['radius'], 3)

    hud_bg = pygame.Rect(0, 0, WIDTH, 40)
    pygame.draw.rect(self.screen, (15, 20, 15), hud_bg)

    ammo_str = "RELOADING..." if self.player.reloadng else f"Ammo: {self.player.ammo}/{self.player.max_ammo}"
    hud = self.font.render(
        f"HP: {self.player.hp} {ammo_str} Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.ki"
        True, (160, 220, 120))
    self.screen.blit(hud, (8, 10))

    if self.game_over:
        ov = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        ov.fill((0, 0, 0, 160))
        self.screen.blit(ov, (0, 0))
        m = self.big_font.render("DEVoured!", True, (180, 40, 40))
        s = self.font.render(f"Wave {self.wave} | Score {self.score} | Press R to Restart", True, (200, 200, 200))
        self.screen.blit(m, (WIDTH // 2 - m.get_width() // 2, HEIGHT // 2 - 40))
        self.screen.blit(s, (WIDTH // 2 - s.get_width() // 2, HEIGHT // 2 + 20))
        pygame.display.flip()

def run(self):
    running = True
    while running:
        running = self.handle_events()
        self.update()

```

```

        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()

```

User prompt: implement Task 4 (Zombie Types) in game.py Create two new zombie variants alongside the standard zombie: Fast Zombie: Smaller size (22x22), yellow color, faster speed (2.8 px/frame), 1 HP. Tank Zombie: Larger size (42x42), dark red color, slower speed (0.8 px/frame), 6 HP. Update the spawn_zombie function to randomly spawn standard, fast, and tank zombies into wave progression with weighted chances (50% normal, 30% fast, 20% tank). Also give comands to record

Response: Here is the complete game.py script with **Task 4 (Zombie Types)** fully integrated alongside Tasks 1–3, followed by the commands to record gameplay.

Updated game.py (Tasks 1–4 Complete)

```

import pygame
import random
import math
import time

WIDTH, HEIGHT = 800, 560
FPS = 60
BG = (30, 35, 25)

class Zombie:
    def __init__(self, x, y, z_type="normal"):
        self.type = z_type
        if z_type == "fast":
            self.rect = pygame.Rect(x, y, 22, 22)
            self.color = (200, 180, 50) # Yellow
            self.hp = 1
            self.speed = 2.8
        elif z_type == "tank":
            self.rect = pygame.Rect(x, y, 42, 42)
            self.color = (140, 50, 50) # Dark Red
            self.hp = 6
            self.speed = 0.8
        else: # normal
            self.rect = pygame.Rect(x, y, 30, 30)
            self.color = (60, 140, 60) # Green
            self.hp = 3
            self.speed = 1.5

        self.wobble = random.uniform(0, 6.28)
        self.frame = 0

    def update(self, player_pos):
        px, py = player_pos
        cx, cy = self.rect.center
        dx, dy = px - cx, py - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist:
            self.rect.x += int(dx / dist * self.speed)
            self.rect.y += int(dy / dist * self.speed)
        self.frame += 1

    def hit(self):
        self.hp -= 1
        return self.hp <= 0

    def draw(self, screen):
        wobble_y = int(math.sin(self.frame * 0.2) * 3)
        draw_rect = self.rect.move(0, wobble_y)
        pygame.draw.rect(screen, self.color, draw_rect, border_radius=5)

        eye_offset = 4 if self.type == "fast" else (8 if self.type == "tank" else 6)
        for ex in [draw_rect.x + eye_offset, draw_rect.x + draw_rect.width - eye_offset - 4]:
            pygame.draw.circle(screen, (200, 40, 40), (ex, draw_rect.y + 8), 3 if self.type == "fast" else 4)

class Barrel:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 28, 28)

```

```

        self.color = (180, 90, 30)
        self.radius = 120

    def draw(self, screen):
        pygame.draw.rect(screen, self.color, self.rect, border_radius=4)
        pygame.draw.rect(screen, (220, 140, 40), self.rect, width=2, border_radius=4)

def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width - 30)
        y = random.randint(0, height - 30)
        rect = pygame.Rect(x, y, 30, 30)
        if not rect.colliderect(player_rect.inflate(margin, margin)):
            # Weighted random selection: 50% normal, 30% fast, 20% tank
            z_type = random.choices(["normal", "fast", "tank"], weights=[0.5, 0.3, 0.2])[0]
            return Zombie(x, y, z_type)

SPEED = 4

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60, 160, 220)
        self.bullets = []
        self.shoot_cooldown = 0
        self.hp = 3
        self.invincible_timer = 0
        self.max_ammo = 12
        self.ammo = 12
        self.reload = False
        self.reload_timer = 0

    def move(self, keys, width, height):
        dx = dy = 0
        if keys[pygame.K_w] or keys[pygame.K_UP]: dy = -SPEED
        if keys[pygame.K_s] or keys[pygame.K_DOWN]: dy = SPEED
        if keys[pygame.K_a] or keys[pygame.K_LEFT]: dx = -SPEED
        if keys[pygame.K_d] or keys[pygame.K_RIGHT]: dx = SPEED
        self.rect.x = max(0, min(width - self.rect.width, self.rect.x + dx))
        self.rect.y = max(0, min(height - self.rect.height, self.rect.y + dy))

        if self.shoot_cooldown > 0:
            self.shoot_cooldown -= 1
        if self.invincible_timer > 0:
            self.invincible_timer -= 1

        if self.reload:
            self.reload_timer -= 1
            if self.reload_timer <= 0:
                self.ammo = self.max_ammo
                self.reload = False

    def reload(self):
        if not self.reload and self.ammo < self.max_ammo:
            self.reload = True
            self.reload_timer = 120

    def take_damage(self):
        if self.invincible_timer == 0:
            self.hp -= 1
            self.invincible_timer = 60
            return True
        return False

    def shoot(self, target_pos):
        if self.shoot_cooldown > 0 or self.reload: return
        if self.ammo <= 0:
            self.reload()
            return

        cx, cy = self.rect.center
        tx, ty = target_pos
        dx, dy = tx - cx, ty - cy
        dist = (dx**2 + dy**2) ** 0.5
        if dist == 0: return
        vx, vy = dx / dist * 10, dy / dist * 10

```

```

        self.bullets.append([cx - 4, cy - 4, vx, vy])
        self.shoot_cooldown = 15
        self.ammo -= 1

    def update_bullets(self, width, height):
        live = []
        for b in self.bullets:
            b[0] += b[2]; b[1] += b[3]
            if 0 <= b[0] <= width and 0 <= b[1] <= height:
                live.append(b)
        self.bullets = live

    def draw(self, screen):
        if self.invincible_timer % 10 < 5:
            pygame.draw.rect(screen, self.color, self.rect, border_radius=6)

        for b in self.bullets:
            pygame.draw.circle(screen, (255, 220, 60), (int(b[0]), int(b[1])), 5)

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Zombie Escape")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 18)
        self.big_font = pygame.font.SysFont("monospace", 44, bold=True)
        self.explosions = []
        self.reset()

    def reset(self):
        self.player = Player(WIDTH // 2, HEIGHT // 2)
        self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
        self.barrels = [
            Barrel(150, 120),
            Barrel(650, 120),
            Barrel(150, 420),
            Barrel(650, 420)
        ]
        self.score = 0
        self.wave = 1
        self.kills = 0
        self.kills_to_next = 8
        self.game_over = False
        self.start_time = time.time()
        self.explosions = []

    def trigger_explosion(self, cx, cy, radius):
        self.explosions.append({'pos': (cx, cy), 'radius': radius, 'duration': 15})
        dead_zombies = []
        for z in self.zombies:
            zx, zy = z.rect.center
            dist = math.hypot(zx - cx, zy - cy)
            if dist <= radius:
                dead_zombies.append(z)

        for z in dead_zombies:
            if z in self.zombies:
                self.zombies.remove(z)
                self.kills += 1
                self.score += 10

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    if self.game_over:
                        self.reset()
                    else:
                        self.player.reload()
                if event.type == pygame.MOUSEBUTTONDOWN and not self.game_over:
                    self.player.shoot(event.pos)
        return True

    def update(self):
        if self.game_over: return
        keys = pygame.key.get_pressed()

```

```

self.player.move(keys, WIDTH, HEIGHT)
self.player.update_bullets(WIDTH, HEIGHT)
self.score = int(time.time() - self.start_time)

self.explosions = [e for e in self.explosions if e['duration'] > 0]
for e in self.explosions:
    e['duration'] -= 1

for z in self.zombies:
    z.update(self.player.rect.center)
    if z.rect.colliderect(self.player.rect):
        if self.player.take_damage():
            if self.player.hp <= 0:
                self.game_over = True

dead = []
barrels_to_remove = []
for b in self.player.bullets[:]:
    bx, by = int(b[0]), int(b[1])

    for barrel in self.barrels:
        if barrel.rect.collidepoint(bx, by):
            if b in self.player.bullets:
                self.player.bullets.remove(b)
            if barrel not in barrels_to_remove:
                barrels_to_remove.append(barrel)
                self.trigger_explosion(barrel.rect.centerx, barrel.rect.centery, barrel.radius)
            break

    for z in self.zombies:
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
                self.player.bullets.remove(b)

for barrel in barrels_to_remove:
    if barrel in self.barrels:
        self.barrels.remove(barrel)

for z in dead:
    if z in self.zombies:
        self.zombies.remove(z)
        self.kills += 1
        self.score += 10

if self.kills >= self.kills_to_next:
    self.kills = 0
    self.wave += 1
    self.kills_to_next = 8 + self.wave * 2
    for _ in range(self.wave + 3):
        self.zombies.append(spawn_zombie(WIDTH, HEIGHT, self.player.rect))

def draw(self):
    self.screen.fill(BG)
    for x in range(0, WIDTH, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (x, 0), (x, HEIGHT), 1)
    for y in range(0, HEIGHT, 60):
        pygame.draw.line(self.screen, (40, 45, 35), (0, y), (WIDTH, y), 1)

    for barrel in self.barrels:
        barrel.draw(self.screen)
    for z in self.zombies:
        z.draw(self.screen)
    self.player.draw(self.screen)

    for e in self.explosions:
        pygame.draw.circle(self.screen, (255, 120, 30), e['pos'], e['radius'], 3)

    hud_bg = pygame.Rect(0, 0, WIDTH, 40)
    pygame.draw.rect(self.screen, (15, 20, 15), hud_bg)

    ammo_str = "RELOADING..." if self.player.reloadng else f"Ammo: {self.player.ammo}/{self.player.max_ammo}"
    hud = self.font.render(
        f"HP: {self.player.hp} {ammo_str} Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.ki"
        True, (160, 220, 120))
    self.screen.blit(hud, (8, 10))

    if self.game_over:

```

```

        ov = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        ov.fill((0, 0, 0, 160))
        self.screen.blit(ov, (0, 0))
        m = self.big_font.render("DEVoured!", True, (180, 40, 40))
        s = self.font.render(f"Wave {self.wave} | Score {self.score} | Press R to Restart", True, (200, 200, 200))
        self.screen.blit(m, (WIDTH // 2 - m.get_width() // 2, HEIGHT // 2 - 40))
        self.screen.blit(s, (WIDTH // 2 - s.get_width() // 2, HEIGHT // 2 + 20))
        pygame.display.flip()

    def run(self):
        running = True
        while running:
            running = self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
        pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()

```

Commands to Record Gameplay

If you need to record screen clips for your submission checklist (e.g., a 10-second gameplay video), you can use built-in CLI screen recording tools depending on your operating system:

macOS (using built-in screencapture)

```
# Record screen interactively (drag box over game window) for 10 seconds
screencapture -v -V 10 output.mp4
```

Linux (using ffmpeg)

```
# Record top-left 800x560 region for 10 seconds
ffmpeg -f x11grab -video_size 800x560 -i :0.0 -t 10 output.mp4
```

Windows (using built-in shortcuts or ffmpeg)

- Press **Win + Alt + R** to start/stop built-in Xbox Game Bar recording.
- Or use ffmpeg via PowerShell:

```
ffmpeg -f gdigrab -framerate 30 -i desktop -t 10 output.mp4
```
